



SAKARYA UYGULAMALI BİLİMLER ÜNİVERSİTESİ

SİSTEM PROGRAMLAMA

3.PROJE ÖDEVİ RAPORU

DERSİN ÖĞRETİM ÜYESİ

PROF. DR. HALİT ÖZTEKİN

HAZIRLAYANLAR

Elif Aytan 22010903035

Lavin Al Najm 23010903014

Merve Say 22010903096

1. GİRİŞ VE LİTERATÜR ARAŞTIRMASI	2
1.1. Gömülü Sistem Yükleme Mimarileri	2
1.1.1.UART Tabanlı Program Yükleme	3
1.1.2.SPI Tabanlı Program Yükleme	3
1.1.3.JTAG Tabanlı Program Yükleme.....	3
1.1.4.Yöntemlerin Karşılaştırılması (Tablo-1).....	3
1.2. Seri Haberleşme ve Veri Doğrulama Protokolleri	4
1.2.1.Checksum Yöntemi	4
1.2.2. CRC (Cyclic Redundancy Check) Yöntemi	5
1.2.3. Bu Projede Checksum Tercih Edilme Nedeni.....	6
2. SİSTEM MİMARİSİ VE DONANIM-YAZILIM ORTAK TASARIMI (CO-DESIGN)	6
2.1. Toolchain Arayüz Standartları	6
2.2. FPGA Loader ve PicoRV32 Bellek Haritası	7
3. DENEYSEL ÇALIŞMALAR, TEST VE ANALİZ	9
3.1. Deney Tasarımı ve Test Senaryoları	9
3.1.1. Test Senaryosu 1: Kara Şimşek Devresi	9
3.1.2. Test Senaryosu 2: Toplama İşlemi	10
3.1.3. Test Senaryosu 3: Buton Sayacı	10
3.2. Veri Toplama ve Donanım Metrikleri	10
3.3. Sonuçların Değerlendirilmesi.....	11
4. PROJENİN KÜRESEL, TOPLUMSAL VE EKONOMİK ETKİLERİ	11
4.1. Sürdürülebilirlik ve Yeşil Bilişim (Green Computing)	11
4.2. Ekonomik Sürdürülebilirlik ve Teknolojik Bağımsızlık	12
4.3. Fonksiyonel Güvenlik ve Sağlık.....	13
4.4. E-Atık Yönetimi ve Döngüsel Ekonomi	14
5. PROJE YÖNETİMİ VE TAKIM ÇALIŞMASI	15
5.1. Görev Dağılımı ve Sorumluluk Matrisi	15
5.2. Koordinasyon ve Sürüm Kontrol Yönetimi	15
6. BİREYSEL KATKI BEYANI.....	15
6.1. Merve Say (Donanım ve FSM Odaklı).....	15

6.2. Lavin Al Najm (Toolchain ve Sistem Yazılımı Odaklı).....	16
6.3. Elif Aytan (Host Yazılımı ve Donanımsal Test Odaklı)	16
7. SONUÇ.....	17
8. KAYNAKÇA.....	19

1. GİRİŞ VE LİTERATÜR ARAŞTIRMASI

Gömülü sistemlerde yazılım geliştirme süreci yalnızca kaynak kodun yazılması ile sınırlı olmayıp, geliştirilen yazılımın hedef donanım üzerine güvenilir ve kontrollü bir şekilde aktarılmasını da kapsamaktadır. Özellikle FPGA tabanlı sistemlerde işlemci çekirdeğinin, bellek yapılarının ve çevre birimlerinin aynı platform üzerinde bulunması, program yükleme süreçlerinin sistem performansı ve güvenilirliği açısından kritik hale gelmesine neden olmaktadır [8]. Bu nedenle günümüzde gömülü sistem tasarımlarında program yükleme mekanizmaları, sistem mimarisinin ayrılmaz bir parçası olarak değerlendirilmektedir.

Bu proje kapsamında, önceki aşamalarda geliştirilen RISC-V RV32I tabanlı assembler ve linker yazılımlarının FPGA üzerinde çalışan bir yükleyici (loader) sistemi ile bütünleştirilmesi amaçlanmıştır. Böylece Assembly dilinde yazılan bir programın makine koduna dönüştürülmesi, bağlanması (linking), FPGA üzerindeki belleğe aktarılması ve PicoRV32 işlemcisi tarafından çalıştırılması süreçlerini kapsayan uçtan uca bir araç zinciri (toolchain) oluşturulmuştur. Geliştirilen sistem; bilgisayar tarafında çalışan Python tabanlı gönderici uygulaması, UART haberleşme altyapısı, FPGA üzerinde çalışan loader modülü, blok bellekler (BRAM) ve PicoRV32 işlemci çekirdeğinden oluşmaktadır [3].

Projenin temel amacı yalnızca çalışan bir sistem geliştirmek değil, aynı zamanda farklı bileşenler arasındaki etkileşimi analiz ederek donanım-yazılım ortak tasarımı yaklaşımını uygulamaktır. Bu kapsamda veri aktarım güvenilirliğini artırmak amacıyla hata kontrol mekanizmaları kullanılmış, FPGA üzerindeki bellek yönetimi sonlu durum makinesi (FSM) yaklaşımı ile gerçekleştirilmiş ve sistem farklı test senaryoları altında değerlendirilmiştir. Ayrıca açık kaynaklı RISC-V mimarisinin sağladığı modülerlik, genişletilebilirlik ve lisans bağımsızlığı avantajları dikkate alınarak maliyet, sürdürülebilirlik ve teknolojik bağımsızlık boyutları da incelenmiştir [1], [9] (PÇ6).

1.1. Gömülü Sistem Yükleme Mimarileri

Gömülü sistemlerde yazılımın hedef donanım üzerine aktarılması ve çalıştırılması işlemi, sistemin güvenilirliği ve sürdürülebilirliği açısından kritik bir öneme sahiptir. Bu amaçla kullanılan yapılar genel olarak bootloader ve loader olarak adlandırılmaktadır. Bootloader, sistem açılışında çalışan ve işletim sistemi ya da uygulama yazılımını yükleyen temel yazılım katmanını ifade ederken; loader ise belirli bir programın veya makine kodunun hedef belleğe aktarılmasından sorumlu olan mekanizmadır. Özellikle FPGA tabanlı sistemlerde loader yapıları, işlemci çekirdeğinin yeniden programlanabilmesi ve sistem güncellemelerinin kolaylaştırılması açısından önemli bir rol üstlenmektedir.

Literatürde program yükleme işlemleri için yaygın olarak UART, SPI ve JTAG tabanlı yöntemler kullanılmaktadır. Bu yöntemlerin her biri farklı performans, maliyet ve uygulama karmaşıklığı özelliklerine sahiptir.

1.1.1.UART Tabanlı Program Yükleme

UART (Universal Asynchronous Receiver Transmitter), asenkron seri haberleşme sağlayan ve gömülü sistemlerde en yaygın kullanılan iletişim yöntemlerinden biridir [5], [10]. UART tabanlı yükleyicilerde bilgisayar tarafından gönderilen program verileri seri port aracılığıyla hedef sisteme aktarılır. FPGA üzerinde çalışan yükleyici modülü gelen verileri belirlenen bellek adreslerine yazar ve yükleme işlemi tamamlandıktan sonra işlemciyi çalıştırır.

UART yönteminin tercih edilmesindeki temel sebepler; düşük donanım maliyeti, kolay uygulanabilirlik, hata ayıklama süreçlerinin basitliği ve neredeyse tüm geliştirme kartlarında yerleşik olarak desteklenmesidir [5]. Buna karşın veri aktarım hızının SPI ve JTAG gibi alternatif yöntemlere göre daha düşük olması önemli bir dezavantaj olarak değerlendirilmektedir.

1.1.2.SPI Tabanlı Program Yükleme

SPI (Serial Peripheral Interface), yüksek hızlı senkron veri iletişimi sağlayan bir haberleşme protokolüdür ve gömülü sistemlerde harici bellek erişimi ile program yükleme uygulamalarında yaygın olarak kullanılmaktadır [6]. FPGA ve mikrodenetleyici sistemlerinde özellikle harici Flash belleklerden program yükleme amacıyla kullanılmaktadır. SPI tabanlı sistemlerde veri aktarımı UART'a göre daha hızlı gerçekleşmekte ve daha yüksek bant genişliği sunmaktadır.

1.1.3.JTAG Tabanlı Program Yükleme

JTAG (Joint Test Action Group), IEEE 1149.1 standardı kapsamında geliştirilen ve özellikle FPGA, ASIC ve mikroişlemci tabanlı sistemlerde kullanılan bir test ve programlama arayüzüdür [4]. JTAG yalnızca program yükleme amacıyla değil, aynı zamanda sistem seviyesinde hata ayıklama ve donanım doğrulama işlemlerinde de kullanılmaktadır.

1.1.4.Yöntemlerin Karşılaştırılması (Tablo-1)

Özellik	UART	SPI	JTAG
Donanım Karmaşıklığı	Düşük	Orta	Yüksek
Veri Aktarım Hızı	Orta	Yüksek	Yüksek
Uygulama Kolaylığı	Yüksek	Orta	Düşük
Hata Ayıklama Kolaylığı	Yüksek	Orta	Çok Yüksek
Maliyet	Düşük	Orta	Yüksek

Tablo 1: Yöntemlerin Karşılaştırılması

Bu proje kapsamında UART tabanlı yükleme mimarisi tercih edilmiştir. Tercihin temel nedenleri arasında düşük maliyet, FPGA üzerinde kolay uygulanabilirlik, hata ayıklama süreçlerinin basitliği ve geliştirilen loader yapısının eğitim amaçlı bir sistem içerisinde hızlı şekilde doğrulanabilmesi yer almaktadır. Ayrıca UART altyapısı, geliştirilen assembler ve linker yazılımlarının çıktılarının FPGA ortamına aktarılması için yeterli veri aktarım kapasitesini sağlamaktadır (PÇ6).

1.2. Seri Haberleşme ve Veri Doğrulama Protokolleri

Gömülü sistemlerde program yükleme işlemlerinin güvenilir şekilde gerçekleştirilebilmesi için yalnızca veri aktarımının sağlanması yeterli değildir. Aktarım sırasında oluşabilecek bit hatalarının tespit edilmesi ve mümkün olduğunca önlenmesi gerekmektedir. Özellikle FPGA tabanlı sistemlerde yükleme sırasında meydana gelen tek bir veri hatası dahi programın yanlış çalışmasına veya sistemin tamamen işlevsiz hale gelmesine neden olabilmektedir. Bu nedenle haberleşme protokollerinde hata kontrol mekanizmalarının kullanılması kritik bir gereklilik olarak kabul edilmektedir [6].

Bu proje kapsamında bilgisayar ile FPGA arasındaki veri aktarımı UART haberleşme protokolü üzerinden gerçekleştirilmiştir. UART yapısı asenkron haberleşme temeline dayandığından, uzun veri transferlerinde gürültü, senkronizasyon kaybı veya fiziksel bağlantı problemleri nedeniyle veri bozulmaları meydana gelebilmektedir [5]. Bu riskleri azaltmak amacıyla literatürde yaygın olarak kullanılan Checksum ve CRC (Cyclic Redundancy Check) yöntemleri incelenmiş ve karşılaştırılmıştır. Aşağıdaki tablo 1.2’de karşılaştırılmıştır.

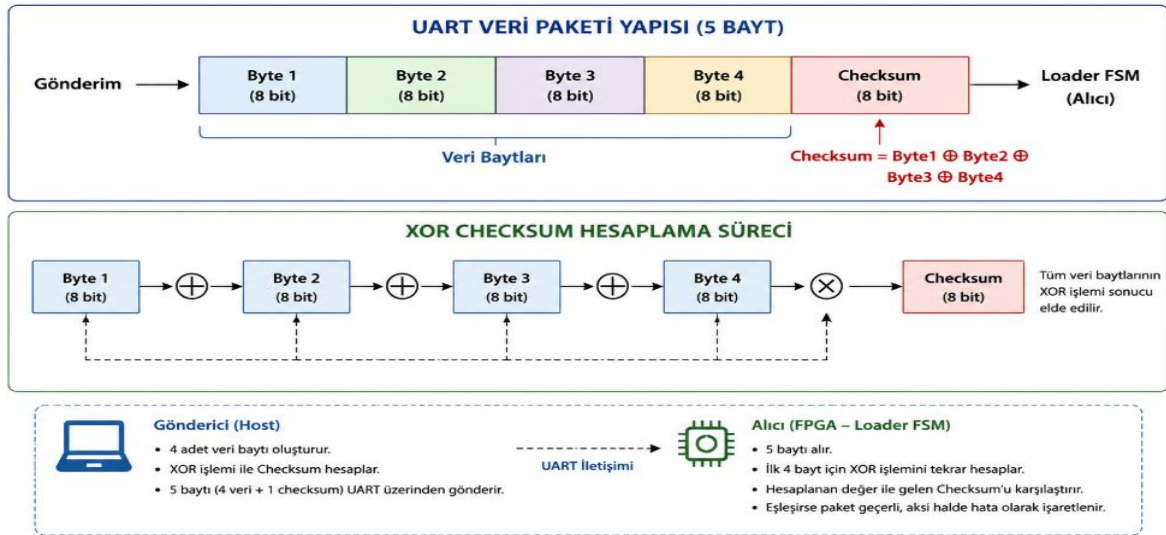
1.2.1.1. Checksum Yöntemi

Checksum, veri paketindeki baytların belirli bir işlemde geçirilerek tek bir doğrulama değeri üretilmesine dayanan basit ve yaygın bir hata tespit yöntemidir [6]. Gönderici tarafında hesaplanan bu değer veri paketinin sonuna eklenmekte, alıcı tarafında ise aynı işlem tekrar uygulanarak sonuçlar karşılaştırılmaktadır. Sonuçların eşleşmesi durumunda veri paketinin doğru şekilde iletilildiği kabul edilmektedir.

Checksum yönteminin temel avantajları aşağıdaki gibidir:

- Hesaplama maliyetinin düşük olması
- Donanım kaynaklarını minimum seviyede kullanması
- FPGA üzerinde kolay uygulanabilmesi
- Gerçek zamanlı sistemlerde düşük gecikme oluşturmaması

Bununla birlikte, çoklu bit hatalarının tespit edilmesinde CRC yöntemine göre daha düşük başarı oranına sahiptir. Bu projede kullanılan Checksum mekanizması Şekil-1’de gösterilmiştir.



Şekil 1: XOR Checksum Mekanizması

Bu çalışmada veri doğrulama yöntemi olarak XOR tabanlı Checksum algoritması kullanılmıştır. Gönderilen her veri paketi dört veri baytı ve bir doğrulama baytından oluşmaktadır. Doğrulama baytı, veri paketindeki tüm veri baytlarının XOR (Exclusive OR) işlemi sonucunda elde edilmektedir.

Checksum değeri aşağıdaki denklem kullanılarak hesaplanmaktadır:

$$\text{Checksum} = \text{Byte}_1 \oplus \text{Byte}_2 \oplus \text{Byte}_3 \oplus \text{Byte}_4 \quad (1)$$



Şekil-2: Üretilen 5 Baytlık UART Paket Yapısı ve XOR Hesaplama Bileşenleri

Burada ⊕ operatörü XOR işlemini ifade etmektedir.

FPGA tarafında Loader FSM tarafından alınan veri paketi için aynı XOR işlemi tekrar uygulanmakta ve hesaplanan değer ile gönderilen Checksum değeri karşılaştırılmaktadır. Sonuçların eşleşmesi durumunda veri paketi geçerli kabul edilerek BRAM belleğine yazılmaktadır. Eşleşmeme durumunda ise paket hatalı olarak işaretlenmekte, yazma işlemi durdurulmakta ve sistem güvenli şekilde IDLE durumuna geri dönmektedir.

XOR Checksum yöntemi; düşük donanım maliyeti, hızlı hesaplanabilmesi ve FPGA üzerinde çok az mantık kaynağı kullanılarak gerçekleştirilebilmesi nedeniyle bu projede tercih edilmiştir. Ayrıca yöntemin basit yapısı, Loader FSM içerisinde gerçek zamanlı doğrulama yapılmasına olanak sağlayarak sistemin güvenilirliğini artırmaktadır.

1.2.2. CRC (Cyclic Redundancy Check) Yöntemi

CRC, veri bütünlüğünü doğrulamak amacıyla kullanılan ve veri paketlerini belirli bir polinom üzerinden işleyerek hata kontrol kodu

oluşturan gelişmiş bir doğrulama mekanizmasıdır [6]. Özellikle Ethernet, USB ve CAN Bus gibi yüksek güvenilirlik gerektiren haberleşme sistemlerinde yaygın olarak kullanılmaktadır.

CRC yönteminin avantajları şunlardır:

- Çoklu bit hatalarını yüksek doğrulukla tespit edebilmesi,
- Endüstriyel haberleşme standartlarında yaygın kullanılması,
- Veri bütünlüğünü daha güçlü şekilde koruması.

Ancak FPGA üzerinde uygulanırken ek mantık kaynakları gerektirmekte ve Checksum yöntemine göre daha karmaşık bir tasarım süreci ortaya çıkarmaktadır.

Checksum ve CRC Karşılaştırması (Tablo-2)

Özellik	Checksum	CRC
Uygulama Karmaşıklığı	Düşük	Yüksek
Donanım Kaynağı Kullanımı	Düşük	Orta-Yüksek
Hesaplama Maliyeti	Düşük	Orta
Hata Tespit Yeteneği	Orta	Yüksek
FPGA Uygulama Kolaylığı	Yüksek	Orta
Gerçek Zamanlı Performans	Yüksek	Orta

Tablo-2: Checksum ve CRC Karşılaştırması

1.2.3. Bu Projede Checksum Tercih Edilme Nedeni

Bu çalışmada veri doğrulama yöntemi olarak Checksum tercih edilmiştir. Bunun temel nedeni geliştirilen sistemin eğitim amaçlı bir FPGA yükleyici mimarisi olması ve veri paketlerinin görece küçük boyutlarda aktarılmasıdır. Checksum yöntemi, UART üzerinden gönderilen makine kodlarının doğruluğunu kontrol etmek için yeterli güvenilirliği sağlarken FPGA üzerindeki mantık kaynaklarının verimli kullanılmasına da olanak tanımaktadır. Ayrıca düşük hesaplama maliyeti sayesinde yükleme sürecine ek gecikme getirmemekte ve sistem tasarımını sadeleştirmektedir. Bu nedenle proje kapsamında performans, donanım maliyeti ve tasarım karmaşıklığı arasındaki denge göz önünde bulundurularak Checksum tabanlı hata kontrol yaklaşımı benimsenmiştir (PÇ6).

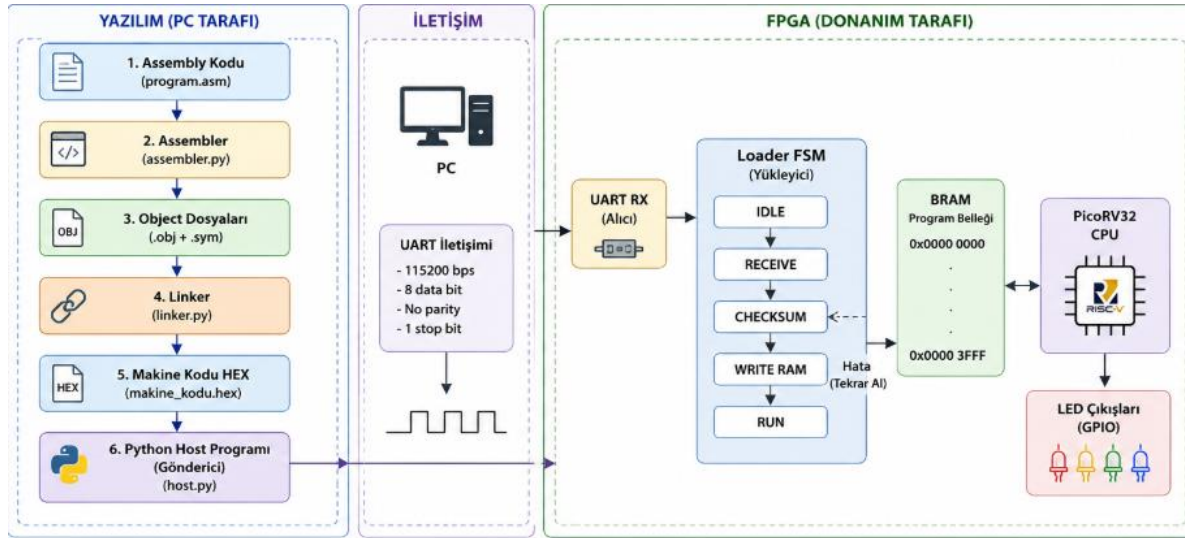
2. SİSTEM MİMARİSİ VE DONANIM-YAZILIM ORTAK TASARIMI (CO-DESIGN)

Bu bölümde geliştirilen FPGA tabanlı yükleyici sisteminin genel mimarisi, yazılım ve donanım bileşenleri arasındaki etkileşim ile veri akış süreçleri açıklanmaktadır. Projede amaç, daha önce geliştirilen assembler ve linker araçlarını FPGA üzerinde çalışan bir yükleyici mekanizması ile bütünleştirerek uçtan uca çalışan bir RISC-V geliştirme ortamı oluşturmaktır. Sistem; bilgisayar tarafında çalışan yazılımlar ile FPGA üzerinde çalışan donanım modüllerinin birlikte çalışmasına dayanan bir donanım-yazılım ortak tasarım (co-design) yaklaşımı ile geliştirilmiştir [3], [8] (PÇ6).

2.1. Toolchain Arayüz Standartları

Geliştirilen sistemde program geliştirme süreci birden fazla yazılım ve donanım bileşeninin koordineli çalışmasıyla gerçekleştirilmektedir. Veri

akışı sırasıyla Assembler, Linker, Host Application, UART haberleşme birimi, Loader modülü ve PicoRV32 işlemcisinden oluşmaktadır. Sistemin genel veri akışı Şekil-3'de gösterilmektedir.



Şekil-3: Genel Sistem Veri Akışı

Assembler modülü, Assembly dilinde yazılan kaynak kodları makine komutlarına dönüştürerek nesne dosyaları oluşturmaktadır. Bu aşamada sembol tabloları oluşturulmakta ve dış referanslar için gerekli bilgiler kayıt altına alınmaktadır.

Linker modülü ise farklı nesne dosyalarını bir araya getirerek global sembolleri çözmekte, relocation işlemlerini gerçekleştirmekte ve FPGA üzerinde çalıştırılabilir son makine kodunu üretmektedir. Üretilen çıktı .hex formatında saklanmaktadır.

Bilgisayar tarafında çalışan Python tabanlı Host Application, linker tarafından oluşturulan .hex dosyasını okuyarak UART üzerinden FPGA kartına göndermektedir [5]. Veri aktarımı sırasında her veri paketi için Checksum değeri hesaplanmakta ve veri paketinin sonuna eklenmektedir. Böylece yükleme sırasında oluşabilecek veri bozulmaları tespit edilebilmektedir.

FPGA üzerinde çalışan Loader modülü ise UART üzerinden gelen verileri alarak doğrulamakta ve belirlenen bellek adreslerine sırasıyla yazmaktadır. Yükleme işlemi tamamlandığında işlemcinin reset hattı serbest bırakılarak programın çalıştırılması sağlanmaktadır (PÇ6).

2.2. FPGA Loader ve PicoRV32 Bellek Haritası

Geliştirilen sistemde PicoRV32 işlemcisi, FPGA içerisinde bulunan blok bellekler (BRAM) ile çalışmaktadır. PicoRV32, açık kaynaklı ve düşük kaynak tüketimine sahip bir RISC-V işlemci çekirdeği olup özellikle FPGA tabanlı eğitim ve araştırma projelerinde yaygın olarak kullanılmaktadır [3].

Program kodları öncelikle Loader modülü tarafından BRAM içerisine yüklenmekte, daha sonra işlemci bu komutları ilgili adreslerden okuyarak yürütmektedir. Bu yapı sayesinde işlemci ile bellek arasındaki erişim

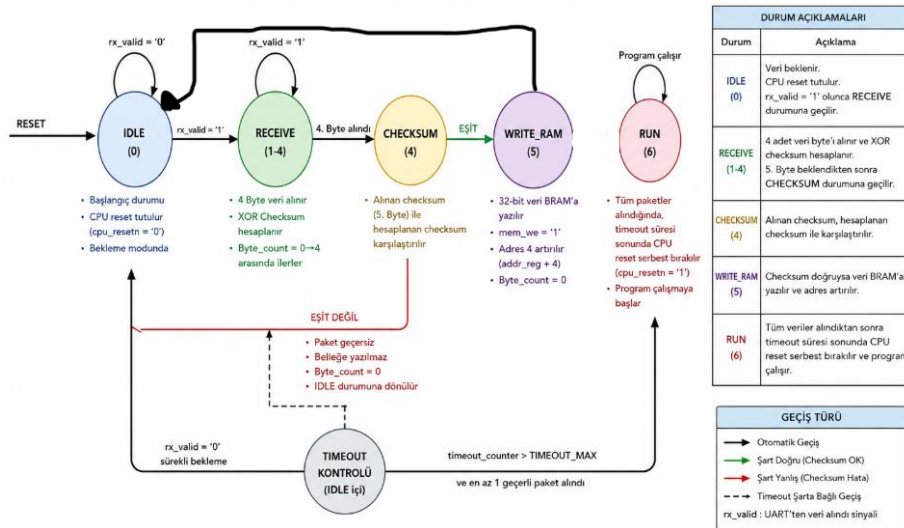
süreçleri sadeleştirilmiş ve sistem kaynaklarının verimli kullanılması sağlanmıştır [7]. Sistemin bellek haritası Tablo-3'te görülmektedir.

Başlangıç Adresi	Bitiş Adresi	Açıklama
0x00000000	0x00000003	Program başlangıç adresi (Reset vektörü)
0x00000004	0x00003FFF	Program komut ve veri depolama alanı (BRAM - 16 KB)
0xFFFFFFF0	0xFFFFFFF3	Buton giriş bölgesi (MMIO - S2 Butonu okuma)
0xFFFFFFF8	0xFFFFFFF8	LED çıkış bölgesi (MMIO - 6-bit LED kontrolü)
0xFFFFFFF8	0xFFFFFFF8	UART TX veri gönderim adresi

Tablo-3: Sistem Bellek Haritası

Loader modülü FPGA üzerinde çalışan sonlu durum makinesi (Finite State Machine - FSM) yapısı kullanılarak gerçekleştirilmiştir. FSM mimarisi veri alma, doğrulama, belleğe yazma ve işlemciyi başlatma işlemlerinin kontrollü şekilde yürütülmesini sağlamaktadır.

Loader modülü FPGA üzerinde UART üzerinden gelen veri paketlerini alan, doğrulayan ve doğrulanan verileri BRAM belleğine yazan bir kontrol mekanizması olarak gerçekleştirilmiştir. Sistem, işlemci program yükleme işlemi tamamlanıncaya kadar PicoRV32 çekirdeğini reset durumunda tutmakta ve yükleme tamamlandığında işlemciyi çalıştırmaktadır. Loader çalışma akışını Şekil-4'te görebilirsiniz.



Şekil-4: Loader Veri Akışı ve Çalışma Mantiği

Sistem açılışında PicoRV32 işlemcisi reset durumunda bekletilmektedir. UART üzerinden veri geldiğinde her dört veri baytı geçici tampon bellekte birleştirilmekte ve aynı anda XOR tabanlı Checksum değeri hesaplanmaktadır. Beşinci bayt olarak gelen Checksum değeri ile hesaplanan değer karşılaştırılmaktadır.

Doğrulama başarılı olduğunda ilgili 32 bitlik makine komutu BRAM belleğine yazılmakta ve bir sonraki bellek adresine geçilmektedir. Doğrulama

başarısız olduğunda ise veri paketi belleğe yazılmamakta ve sistem yeni veri paketini beklemektedir.

UART üzerinden belirli bir süre boyunca yeni veri alınmaması durumunda yükleme işleminin tamamlandığı kabul edilmekte ve PicoRV32 işlemcisinin reset hattı serbest bırakılmaktadır. Böylece işlemci 0x00000000 adresinden başlayarak yüklenen programı yürütmeye başlamaktadır.

Bu yaklaşım sayesinde yalnızca doğrulanmış veri paketleri belleğe yazılmakta, veri aktarımı sırasında oluşabilecek hataların program belleğini bozması önlenmekte ve yükleme işleminin güvenilirliği artırılmaktadır (PÇ6).

Loader modülünün çalışma adımları aşağıda özetlenmiştir:

- **Veri Alma:** UART üzerinden gelen dört veri baytı geçici tampon bellekte birleştirilir.
- **Checksum Hesaplama:** Alınan dört veri baytı için XOR tabanlı Checksum değeri oluşturulur.
- **Doğrulama:** Hesaplanan Checksum ile paketin sonundaki Checksum baytı karşılaştırılır.
- **Belleğe Yazma:** Doğrulama başarılı ise veri BRAM belleğine yazılır ve adres değeri artırılır.
- **Yükleme Tamamlama:** Belirli süre boyunca yeni veri alınmazsa program yükleme işleminin tamamlandığı kabul edilir.
- **Programı Çalıştırma:** PicoRV32 reset durumundan çıkarılır ve program yürütülmeye başlanır.

3. DENEYSEL ÇALIŞMALAR, TEST VE ANALİZ

Bu bölümde geliştirilen FPGA tabanlı yükleyici sisteminin doğruluğunu ve kararlılığını değerlendirmek amacıyla gerçekleştirilen test çalışmaları sunulmaktadır. Test senaryoları hazırlanırken yalnızca programın çalışıp çalışmadığını gözlemlemek yerine; işlemci, yükleyici ve haberleşme altyapısının birlikte doğru şekilde çalışıp çalışmadığını doğrulayacak deneyler tasarlanmıştır. Böylece sistem farklı zorluk seviyelerindeki uygulamalar altında değerlendirilmiş ve elde edilen sonuçlar analiz edilmiştir (PÇ7).

3.1. Deney Tasarımı ve Test Senaryoları

Geliştirilen FPGA tabanlı yükleyici sisteminin doğruluğunu değerlendirmek amacıyla üç farklı Assembly test senaryosu hazırlanmıştır. Test senaryoları seçilirken yalnızca işlemci komutlarının doğruluğu değil, aynı zamanda assembler, linker, loader, UART haberleşmesi ve FPGA çevre birimlerinin birlikte çalışması da değerlendirilmiştir. Böylece sistem farklı kullanım senaryoları altında doğrulanmıştır (PÇ7).

3.1.1. Test Senaryosu 1: Kara Şimşek Devresi

Bu test senaryosunda FPGA kartı üzerinde bulunan adreslenebilir LED'lerin işlemci tarafından kontrol edilmesi amaçlanmıştır. Program içerisinde LED çıkış adresine farklı veri desenleri yazılarak LED'lerin sağa ve sola kayması sağlanmıştır. Ayrıca ana program ile ayrı dosyada tanımlanan gecikme fonksiyonu arasında fonksiyon çağrısı gerçekleştirilmiştir.

Amaçlanan doğrulamalar:

- Global sembollerin linker tarafından doğru çözülmesi
- Relocation işlemlerinin doğrulanması
- JAL ve JALR komutlarının doğru çalışması
- Bellek eşlemeli giriş/çıkış (Memory Mapped I/O) yapısının doğrulanması
- FPGA üzerindeki LED çıkışlarının kontrol edilmesi

Test sonucunda LED'lerin belirlenen desen doğrultusunda sıralı olarak hareket ettiği ve işlemcinin fonksiyon çağrılarından doğru şekilde geri dönebildiği gözlemlenmiştir.

3.1.2. Test Senaryosu 2: Toplama İşlemi

Bu test senaryosunda işlemcinin aritmetik işlem yetenekleri ve UART haberleşme altyapısı test edilmiştir. Program içerisinde iki sayı toplanmış ve elde edilen sonuç ASCII karakterlerine dönüştürülerek UART üzerinden bilgisayara gönderilmiştir.

Amaçlanan doğrulamalar:

- • ADD komutunun doğru çalışması
- • Registerlar arasında veri aktarımının doğrulanması
- • UART haberleşme mekanizmasının doğrulanması
- • Loader tarafından yüklenen programın doğru çalıştırılması
- • Sonucun ASCII karakterlerine dönüştürülmesinin doğrulanması

Test sonucunda hesaplanan toplama sonucu seri haberleşme hattı üzerinden başarıyla bilgisayara aktarılmıştır.

3.1.3. Test Senaryosu 3: Buton Sayacı

Bu test senaryosunda FPGA üzerindeki buton girişleri ile LED çıkışlarının birlikte çalışması test edilmiştir. Program sürekli olarak buton durumunu izlemiş, her buton basışında bir sayaç değeri artırılmış ve elde edilen değer LED çıkışlarına yazılmıştır. Ayrıca her basış olayında UART üzerinden bilgi gönderilmiştir.

Amaçlanan doğrulamalar:

- Bellek eşlemeli giriş birimlerinin doğrulanması
- FPGA üzerindeki butonların okunabilmesi
- Sayaç işlemlerinin doğru çalışması
- LED çıkışlarının güncellenmesi
- UART üzerinden olay bildiriminin yapılması

Test sonucunda her buton basışında sayaç değerinin arttığı, LED göstergesinin güncellendiği ve sistemin kararlı şekilde çalıştığı gözlemlenmiştir.

3.2. Veri Toplama ve Donanım Metrikleri

Sistemin performansını değerlendirmek amacıyla farklı boyutlardaki makine kodları FPGA üzerine yüklenmiş ve yükleme süreleri ölçülmüştür. Ayrıca

sentez işlemleri sonucunda elde edilen donanım kaynak kullanım raporları incelenmiştir. Kod boyutlarına göre yükleme süreleri Tablo-4'te ve kaynak kullanımları da Tablo-5'te verilmiştir.

Test Programı	Kod Boyutu(Byte)	Yükleme Süresi(sn)
Test1	116 Byte	0.05 sn
Test2	152 Byte	0.07 sn
Test3	144 Byte	0.06 sn

Tablo-4: Kod Boyutuna Göre Yükleme Süreleri

Kaynak Türü	Kullanılan Miktar	Toplam Kapasite	Kullanım Oranı (%)
LUT (Look-Up Table)	1465	8640	%16,9
Total Logic (Mantık Elemanı)	1829	8640	%22,0
Register (Yazmaç / FF)	795	6693	%12,0
BRAM (Block RAM)	10 Blok (8 SP + 2 SDPB)	26 Blok	%39,0
IO Buffer (Giriş/Çıkış Pinleri)	11	71	%16,0

Tablo-5: FPGA Kaynak Kullanım Raporu

3.3. Sonuçların Değerlendirilmesi

Gerçekleştirilen deneyler sonucunda geliştirilen yükleyici sisteminin farklı karmaşıklık seviyelerindeki Assembly programlarını başarıyla yükleyebildiği ve PicoRV32 işlemcisi üzerinde çalıştırabildiği doğrulanmıştır. Ayrıca UART tabanlı veri aktarım mekanizmasının Checksum doğrulama yapısı ile kararlı şekilde çalıştığı gözlemlenmiştir. Yapılan testler sistemin yalnızca teorik olarak değil, fiziksel FPGA donanımı üzerinde de başarılı şekilde çalışabildiğini göstermektedir (PÇ7).

4. PROJENİN KÜRESEL, TOPLUMSAL VE EKONOMİK ETKİLERİ

Mühendislik projelerinin değerlendirilmesinde yalnızca teknik başarı kriterleri yeterli değildir. Günümüzde geliştirilen sistemlerin çevresel etkileri, ekonomik sürdürülebilirlikleri, toplumsal katkıları ve güvenlik boyutları da önemli değerlendirme ölçütleri arasında yer almaktadır. Bu proje kapsamında geliştirilen FPGA tabanlı PicoRV32 yükleyici sistemi, açık kaynaklı RISC-V mimarisini temel alması ve yeniden programlanabilir donanım yapısını kullanması nedeniyle sürdürülebilirlik, teknolojik bağımsızlık ve güvenilirlik açısından çeşitli avantajlar sunmaktadır (PÇ8).

4.1. Sürdürülebilirlik ve Yeşil Bilişim (Green Computing)

Bilgi teknolojileri alanında enerji verimliliği ve kaynak kullanımının optimize edilmesi, sürdürülebilir bilişim anlayışının temel hedeflerinden biridir. Geleneksel işlemci mimarilerinde kullanılan karmaşık komut setleri

daha fazla donanım kaynağı ve enerji tüketimi gerektirebilmektedir. Buna karşılık RISC-V mimarisi sade ve modüler yapısı sayesinde daha düşük donanım karmaşıklığı ile gerçekleştirilebilmektedir [1], [9].

Bu proje kapsamında kullanılan PicoRV32 çekirdeği, minimal kaynak tüketimi ile çalışan hafif bir RISC-V işlemcisidir [3]. FPGA üzerinde gerçekleştirilen tasarımda yalnızca gerekli modüller kullanılmış, böylece mantık elemanı ve bellek kullanımının azaltılması hedeflenmiştir.

Proje kapsamında geliştirilen UART tabanlı loader yapısı sayesinde oluşturulan yeni programlar FPGA kartı değiştirilmeden tekrar yüklenebilmektedir. Örneğin proje sürecinde geliştirilen Kara Şimşek Devresi, Toplama İşlemi ve Buton Sayacı test senaryoları aynı donanım üzerinde yalnızca yeni .hex dosyalarının yüklenmesiyle çalıştırılmıştır. Böylece yeni bir donanım üretimine veya fiziksel değişikliğe ihtiyaç duyulmamıştır.

Ayrıca yazılımın FPGA içerisine yeniden yüklenebilmesi sayesinde donanım değişikliğine ihtiyaç duyulmadan sistem güncellenebilmekte ve kaynak tüketimi azaltılmaktadır. Bu yaklaşım Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçları kapsamında yer alan SKA 7 (Erişilebilir ve Temiz Enerji) ile SKA 13 (İklim Eylemi) hedeflerini desteklemektedir (PÇ8).

4.2. Ekonomik Sürdürülebilirlik ve Teknolojik Bağımsızlık

Son yıllarda yarı iletken teknolojilerindeki gelişmeler, ülkelerin teknolojik bağımsızlığını stratejik bir konu haline getirmiştir. Ticari işlemci mimarileri genellikle lisans maliyetleri ve kapalı kaynak yapıları nedeniyle geliştirme süreçlerinde çeşitli kısıtlamalar oluşturmaktadır.

RISC-V mimarisi ise açık kaynaklı bir komut seti mimarisi olması sayesinde araştırmacılara, üniversitelere ve yerli teknoloji firmalarına özgür geliştirme imkânı sunmaktadır [1]. Bu özellikleri sayesinde RISC-V günümüzde üniversiteler, araştırma kuruluşları ve yarı iletken sektöründe faaliyet gösteren firmalar tarafından yaygın olarak kullanılmaktadır [1], [2].

Bu proje kapsamında assembler, linker ve loader yazılımlarının tamamı ekip tarafından geliştirilmiştir. Programların makine koduna dönüştürülmesi, farklı nesne dosyalarının birleştirilmesi ve FPGA belleğine yüklenmesi süreçleri dışa bağımlı ticari araçlar kullanılmadan gerçekleştirilmiştir. Bu durum öğrencilerin işlemci mimarisi, sistem yazılımları ve gömülü sistemler konularında uygulamalı bilgi edinmesine katkı sağlamıştır.

Şekil-5'de proje geliştirme sürecinde kullanılan Gowin FPGA geliştirme ortamına ait lisans bilgileri gösterilmektedir. Eğitim amaçlı sağlanan lisans desteği sayesinde FPGA tasarımı, sentezleme, yerleştirme-yönlendirme ve programlama işlemleri herhangi bir ek ticari maliyet oluşturmada gerçekleştirilebilmiştir. Bu durum öğrencilerin ve araştırmacıların düşük maliyetle FPGA tabanlı sistemler geliştirebilmesine katkı sağlamakta ve Ar-Ge süreçlerinin ekonomik sürdürülebilirliğini desteklemektedir.

```

1  #-----
2  # GOWIN LICENSE
3  #
4  # Copyright (C) GOWINSEMI Corp.
5  #
6  # CREATED AT: 2026-05-19T10:19:35
7  # GRANT TO: gowin
8  #
9  # WARNING: This file must not be edited in any way.
10 #-----
11
12 MODE = NODELOCK
13 TYPE = STD
14 HOST_ID = C01850927ED8
15 EXP_DATE = 2027-05-19
16
17 ----- BEGIN SIGN -----
18 5849e7e7b869fc10a19974ce914f1639570676a4dab9a252f08afa64ef9e2a30
19 b459d334b9e6c68642dcca22fe460b15bb2c93cb76478160da53ac4aabc01049
20 3f837f06aa76165728c47b4581586338f037e2a6803a6b5a4f70f3d4aca19231
21 904259d3f7668e305480e99a1f4f4ee0f31cb1f5ff137c6334eff5b52cb6364b
22 ----- END SIGN -----
23

```

Şekil-5: Gowin FPGA Geliştirme Ortamı Lisans Bilgileri

Ayrıca proje boyunca geliştirilen Kara Şimşek Devresi, Toplama İşlemi ve Buton Sayacı uygulamaları aynı Tang Nano 9K FPGA kartı üzerinde çalıştırılmıştır. Her uygulama için yeni bir donanım geliştirmek yerine yalnızca farklı .hex dosyaları yüklenerek sistem yeniden programlanmıştır. Böylece geliştirme maliyetleri azaltılmış ve mevcut donanım kaynaklarının daha verimli kullanılması sağlanmıştır.

Bu yaklaşım SKA 8 (İnsana Yakışır İş ve Ekonomik Büyüme) ile SKA 9 (Sanayi, Yenilikçilik ve Altyapı) hedefleriyle uyumludur (PÇ8).

4.3. Fonksiyonel Güvenlik ve Sağlık

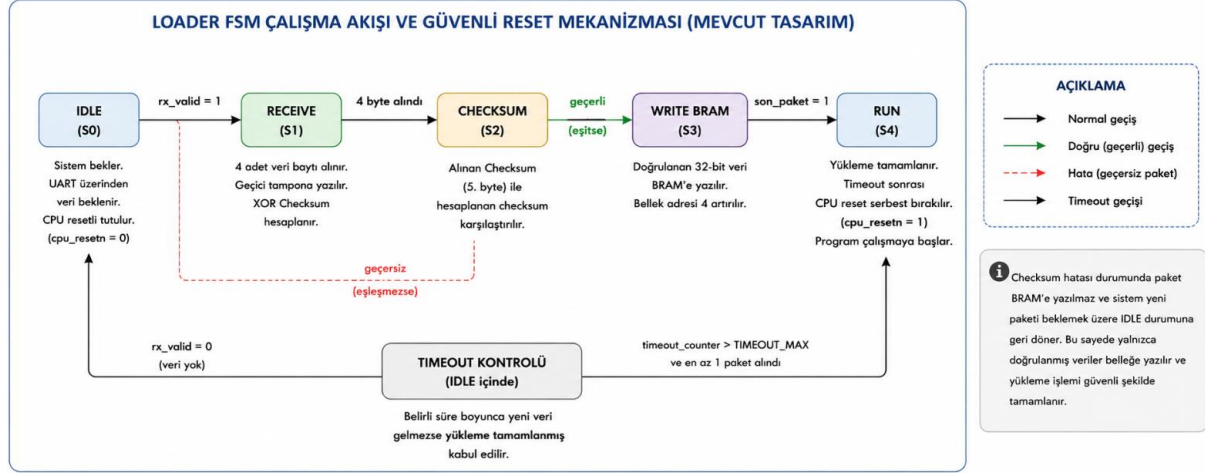
Gömülü sistemler günümüzde sağlık teknolojileri, otomotiv sistemleri, endüstriyel otomasyon ve savunma sanayii gibi kritik alanlarda yaygın olarak kullanılmaktadır [8]. Bu sistemlerde meydana gelebilecek veri aktarım hataları ciddi güvenlik riskleri oluşturabilmektedir.

Bu proje kapsamında geliştirilen yükleyici sistemde UART üzerinden aktarılan verilerin doğruluğu XOR tabanlı Checksum mekanizması ile kontrol edilmektedir. Gönderilen her veri paketi FPGA üzerinde yeniden hesaplanmakta ve doğrulama sonucunda yalnızca geçerli verilerin belleğe yazılmasına izin verilmektedir.

Proje kapsamında kullanılan loader FSM yapısı, veri aktarımı tamamlanmadan işlemcinin çalışmasını engellemektedir. Böylece eksik veya hatalı yüklenmiş programların yürütülmesi önlenmektedir. Ayrıca veri aktarımı tamamlandıktan sonra işlemci kontrollü şekilde başlatılmakta ve program güvenli biçimde çalıştırılmaktadır.

Bu yaklaşım sistem güvenilirliğini artırmakta ve kritik uygulamalarda oluşabilecek risklerin azaltılmasına katkı sağlamaktadır. Özellikle tıbbi cihazlar, ulaşım sistemleri ve savunma uygulamalarında güvenilir yazılım

yükleme mekanizmaları insan hayatı açısından büyük önem taşımaktadır. Bu nedenle geliştirilen yapının fonksiyonel güvenlik açısından önemli bir katkı sunduğu değerlendirilmektedir. Bu durum SKA 3 (Sağlık ve Kaliteli Yaşam) hedefi ile ilişkilendirilebilir (PÇ8).



Şekil-6: Hata Anında Loader Koruma ve Güvenli Reset Mekanizması

4.4. E-Atık Yönetimi ve Döngüsel Ekonomi

Elektronik atıklar günümüzde çevresel sürdürülebilirlik açısından önemli problemlerden biri olarak kabul edilmektedir. Geleneksel sistemlerde yazılım güncellemeleri çoğu zaman yeni donanım gereksinimlerine yol açabilmekte veya cihazların kullanım ömürlerini kısaltabilmektedir.

FPGA tabanlı sistemlerde ise donanım fiziksel olarak değiştirilmeden yeni yazılımlar yüklenebilmekte ve sistem işlevleri güncellenebilmektedir. Bu proje kapsamında geliştirilen loader mekanizması sayesinde FPGA üzerindeki programlar yeniden yüklenebilmekte ve sistem farklı uygulamalara uyarlanabilmektedir.

Proje sürecinde geliştirilen Kara Şimşek Devresi, Toplama İşlemi ve Buton Sayacı uygulamaları aynı Tang Nano 9K FPGA kartı üzerinde çalıştırılmıştır. Her uygulama için yeni bir donanım tasarlamak yerine yalnızca ilgili .hex dosyası UART üzerinden FPGA belleğine yüklenmiş ve sistem yeni görevine uyarlanmıştır. Bu durum geliştirilen loader yapısının donanımın yeniden kullanılmasına olanak sağladığını göstermektedir.

Ayrıca sistemde kullanılan UART tabanlı yüklem mekanizması sayesinde FPGA üzerindeki programlar fiziksel müdahaleye gerek duyulmadan güncellenebilmektedir. Bu yaklaşım, günümüzde yaygın olarak kullanılan uzaktan güncelleme sistemlerinin temel çalışma mantığıyla benzerlik göstermektedir. Böylece geliştirilen altyapı gelecekte OTA (Over-The-Air) güncelleme sistemlerine uyarlanabilecek bir temel sunmaktadır.

Bu yaklaşım donanımın kullanım ömrünü uzatmakta, elektronik bileşenlerin yeniden değerlendirilmesine olanak sağlamakta ve elektronik atık miktarının azaltılmasına katkıda bulunmaktadır. Böylece döngüsel ekonomi anlayışına katkı sağlanmakta ve SKA 12 (Sorumlu Üretim ve Tüketim) hedefleri desteklenmektedir (PÇ8).

5. PROJE YÖNETİMİ VE TAKIM ÇALIŞMASI

Bu proje, birden fazla yazılım ve donanım bileşeninin birlikte geliştirilmesini gerektiren kapsamlı bir sistem entegrasyonu çalışmasıdır. Assembler, linker, UART haberleşmesi, FPGA üzerindeki loader modülü, PicoRV32 işlemcisi ve test süreçlerinin birbirleriyle uyumlu şekilde çalışabilmesi için planlı bir görev dağılımı ve düzenli koordinasyon mekanizmaları uygulanmıştır. Proje süresince takım üyeleri arasında iş bölümü yapılmış, ara entegrasyon toplantıları gerçekleştirilmiş ve modüller aşamalı olarak birleştirilerek sistem doğrulaması yapılmıştır (PÇ12, PÇ13).

5.1. Görev Dağılımı ve Sorumluluk Matrisi

Proje kapsamında gerçekleştirilen çalışmalar ekip üyeleri arasında dengeli şekilde dağıtılmıştır. Görevler belirlenirken her üyenin sistemin farklı aşamalarında aktif rol alması hedeflenmiştir.

Tablo 5.1. Görev Dağılımı

Görev dağılımı hazırlanırken sistemin tüm bileşenlerinin en az iki kişi tarafından incelenmesi sağlanmış ve olası entegrasyon problemlerinin önüne geçilmesi amaçlanmıştır. Bu yaklaşım bilgi paylaşımını artırmış ve proje süresince ortaya çıkan teknik problemlerin daha hızlı çözülmesine katkı sağlamıştır (PÇ13).

5.2. Koordinasyon ve Sürüm Kontrol Yönetimi

Proje geliştirme sürecinde modüller bağımsız olarak geliştirilmiş, daha sonra belirli entegrasyon aşamalarında bir araya getirilmiştir. Özellikle assembler, linker ve loader modülleri arasında kullanılan veri formatlarının uyumlu olması için düzenli koordinasyon toplantıları gerçekleştirilmiştir.

Çalışmalar sırasında her modülün ara çıktıları ekip üyeleri tarafından karşılıklı olarak test edilmiş ve hata durumlarında ortak çözüm süreçleri uygulanmıştır. UART üzerinden veri aktarımı, Checksum doğrulaması ve bellek yönetimi gibi kritik bileşenler birden fazla senaryo altında tekrar tekrar doğrulanmıştır.

Sistem geliştirme sürecinde modüler tasarım yaklaşımı benimsenmiş ve her bileşen bağımsız olarak test edildikten sonra genel sisteme entegre edilmiştir. Böylece hata ayıklama süreçleri kolaylaştırılmış, entegrasyon kaynaklı problemlerin tespiti hızlandırılmış ve sistem kararlılığı artırılmıştır.

Bu yaklaşım sayesinde proje yalnızca teknik açıdan değil, takım koordinasyonu ve proje yönetimi açısından da başarılı bir şekilde yürütülmüştür (PÇ12, PÇ13).

6. BİREYSEL KATKI BEYANI

6.1. Merve Say (Donanım ve FSM Odaklı)

Projede Üstlenilen Spesifik Modüller:

Sistem mimarisinin donanımsal omurgasını oluşturan system_top.vhd ana modülünün tasarımı, loader_fsm.vhd sonlu durum makinesinin ardışıl mantıkla modellenmesi ve açık kaynaklı PicoRV32 RISC-V çekirdeğinin sisteme entegrasyonu tamamen tarafımdan gerçekleştirilmiştir. Ayrıca bilgisayardan gelen 5 baytlık paketlerin donanım tarafında ayrıştırılması ve XOR tabanlı

Checksum doğrulama mantığının VHDL mimarisine uyarlanması görevlerini üstlendim.

Karşılaşılan En Büyük Teknik Problem ve Çözümü:

Entegrasyon aşamasında karşılaştığım en büyük problem, PicoRV32 işlemcisinin bellek talep sinyalleri (mem_valid) ile FPGA'in yerleşik Blok RAM (BRAM) zamanlaması arasındaki senkronizasyon uyumsuzluğu idi. İşlemci bellekten veri okumaya çalışırken BRAM'in bir çevrim gecikmeli yanıt vermesi sistemin kilitlenmesine (trap) yol açıyordu. Bu problemi çözmek için system_top.vhd içerisinde iki çevrimlik bir Zamanlama Pulse Handshake (cpu_mem_ready kontrol döngüsü) mekanizması tasarladım. Bellek erişim talebi geldiğinde hazır sinyalini kontrollü olarak bir clock cycle geciktirerek işlemci ile RAM arasındaki veri hattını kararlı hale getirdim.

İmza:



6.2. Lavin Al Najm (Toolchain ve Sistem Yazılımı Odaklı)

Projede Üstlenilen Spesifik Modüller:

Assembly dilinde yazılan kaynak kodları nesne dosyasına (.obj) dönüştüren Assembler modülü ile bu nesne dosyalarını birleştirerek kesin bellek adreslerini çözen Linker yazılımlarının (kodlar.py içerisindeki altyapı) geliştirilmesi tamamen tarafımdan yürütülmüştür. Nesne dosyası formatı için özgün başlık (Header), Tanımlama (Define), Referans (Reference) ve Relocation (M) kayıt standartlarını belirleyerek toolchain mimarisini inşa ettim.

Karşılaşılan En Büyük Teknik Problem ve Çözümü:

Geliştirme sürecinde karşılaştığım en karmaşık problem, main3.asm ve fonksiyon3.asm gibi iki ayrı dosyada yer alan bağımsız kod blokları birleştirilirken, ileriye/geriye doğru yapılan fonksiyon çağrılarının (jal, bne komutları) göreceli (relative) offset adreslerinin Linker tarafından yanlış hesaplanmasıydı. Bu durum dallanma komutlarının işlemciyi tamamen alakasız hafıza adreslerine fırlatmasına neden oluyordu. Bu problemi aşmak için Linker modülü içerisine İki Geçişli (Two-Pass) Relocation algoritması ekledim. İlk geçişte tüm global sembollerin kesin adreslerini çıkartıp global_symbol.txt tablosuna kaydettim; ikinci geçişte ise kuyruktaki M kayıtlarını tarayarak dallanma komutlarının içindeki 12-bit ve 20-bitlik anlık (immediate) verileri ikiye tümleyen (2's complement) formatında dinamik olarak yeniden hesaplayıp makine koduna (makine_kodu.hex) hatasız enjekte ettim.

İmza:



6.3. Elif Aytan (Host Yazılımı ve Donanımsal Test Odaklı)

Projede Üstlenilen Spesifik Modüller:

Bilgisayar tarafında çalışan, üretilen nihai hex dosyasını okuyarak UART üzerinden 5 baytlık paketler halinde FPGA'e aktaran Python tabanlı Host Uygulamasının (host.py) yazılması tarafımdan gerçekleştirilmiştir. Ayrıca sistemin doğrulanması için kritik önemi haiz olan Kara Şimşek (main.asm), Dinamik Toplama ve Seri Aktarım (main2.asm) ve Donanımsal Buton Sayacı (main3.asm) olmak üzere 3 farklı karmaşık Assembly test senaryosunun yazılması ve donanım portlarının test edilmesi sorumluluğunu üstlendim.

Karşılaşılan En Büyük Teknik Problem ve Çözümü:

Sistemi fiziksel donanım (Tang Nano 9K) üzerinde test ederken karşılaştığım en büyük problem, buton giriş birimindeki mekanik esnemelerden kaynaklanan sinyal arkları (bounce) ve RAM bellek adreslerinin çevre birimleriyle çakışmasıydı. Butona bir kez basılmasına rağmen işlemci bunu onlarca kez basılmış gibi algılıyor ve sayaç hatalı artıyordu. Bu problemi çözmek için fonksiyon3.asm kodunda hem asenkron buton girişini iki aşamalı fliplop üzerinden senkronize ettim (btn_sync_0 ve btn_sync_1), hem de yazılımsal bir Debounce (Ark Sönümleme) döngüsü kurguladım. Butonun basılma ve bırakılma anlarına 500×500 çevrimlik kararlı gecikme döngüleri ekleyerek mekanik gürültüleri tamamen filtreledim. Ayrıca bellek çakışmalarını önlemek için MMIO adreslerini 0xFFFFF80 gibi güvenli yüksek hafıza bölgesine taşıdım.

İmza:



7. SONUÇ

Bu proje kapsamında, önceki aşamalarda geliştirilen RISC-V RV32I tabanlı assembler ve linker yazılımları FPGA üzerinde çalışan bir yükleyici sistemi ile bütünleştirilerek uçtan uca çalışan bir gömülü sistem geliştirme ortamı oluşturulmuştur. Geliştirilen yapı sayesinde Assembly dilinde yazılan programların nesne dosyalarına dönüştürülmesi, bağlanması (linking), UART üzerinden FPGA kartına aktarılması ve PicoRV32 işlemcisi üzerinde çalıştırılması başarıyla gerçekleştirilmiştir.

Proje sürecinde UART tabanlı veri aktarım mekanizması kullanılmış, veri bütünlüğünün korunması amacıyla Checksum doğrulama yöntemi uygulanmış ve FPGA üzerinde çalışan Loader FSM yapısı ile programların belleğe güvenilir şekilde yüklenmesi sağlanmıştır. Ayrıca geliştirilen sistem, assembler, linker, host uygulaması ve FPGA üzerindeki donanım modüllerinin birlikte çalışmasını gerektiren kapsamlı bir donanım-yazılım ortak tasarım çalışması niteliği taşımaktadır.

Gerçekleştirilen test senaryoları sonucunda fonksiyon çağrıları, döngü yapıları, sembol çözümleme işlemleri, relocation mekanizmaları ve bellek eşlemeli giriş/çıkış işlemlerinin doğru şekilde çalıştığı doğrulanmıştır. Yapılan deneyler sistemin yalnızca teorik olarak değil, fiziksel FPGA donanımı üzerinde de uygulanabilir olduğunu göstermiştir.

Bu çalışma sayesinde açık kaynaklı RISC-V mimarisi kullanılarak düşük maliyetli ve esnek bir program yükleme altyapısı oluşturulmuş, öğrencilerin sistem yazılımları, FPGA tasarımı, haberleşme protokolleri ve gömülü sistem entegrasyonu konularında uygulamalı deneyim kazanmaları sağlanmıştır.

Gelecekte yapılabilecek çalışmalar kapsamında sistemin CRC tabanlı hata kontrol mekanizmaları ile geliştirilmesi, SPI tabanlı yüksek hızlı yükleme desteğinin eklenmesi, uzaktan güncelleme (OTA) altyapısının oluşturulması ve daha gelişmiş RISC-V işlemci çekirdekleri ile uyarlanması planlanabilir. Böylece geliştirilen sistem daha yüksek performanslı ve endüstriyel uygulamalara uygun bir yapıya dönüştürülebilecektir.

8. KAYNAKÇA

- [1] RISC-V International, *The RISC-V Instruction Set Manual Volume I: Unprivileged ISA*, Version 20240411, RISC-V Foundation, 2024.
- [2] C. Celio, D. Patterson, and K. Asanović, "The Berkeley Out-of-Order Machine (BOOM): An Open-Source Out-of-Order RISC-V Core," *EECS Department, University of California, Berkeley*, Tech. Rep., 2015.
- [3] C. Wolf, "PicoRV32 - A Size Optimized RISC-V CPU," GitHub Repository, 2024. Available: <https://github.com/YosysHQ/picorv32>
- [4] IEEE Standards Association, *IEEE Standard for Test Access Port and Boundary-Scan Architecture*, IEEE Std 1149.1-2013, 2013.
- [5] J. Axelson, *Serial Port Complete: COM Ports, USB Virtual COM Ports, and Ports for Embedded Systems*, 2nd ed., Lakeview Research, 2007.
- [6] W. Stallings, *Data and Computer Communications*, 11th ed., Pearson Education, 2019.
- [7] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with VHDL Design*, 3rd ed., McGraw-Hill, 2009.
- [8] M. Barr and A. Massa, *Programming Embedded Systems*, 2nd ed., O'Reilly Media, 2006.
- [9] D. Patterson and J. Hennessy, *Computer Organization and Design RISC-V Edition*, 2nd ed., Morgan Kaufmann, 2021.
- [10] Xilinx Inc., *UART Lite v2.0 Product Guide*, Product Specification, 2023.